ADVANCED PROGRAMMING (OOP) LAB PROJECT

Name – Baivabi Chakraborty

IOTCSBT Roll 17

ENROLLMENT NO:-12024052017034

Instructors: Dr. Swagatam Basu,

Dr. Susovan Jana, Dr. Soumadip Biswas

PCCCS405 : Advanced Programming(OOP)

Spring 2026

Apartment Management System using Object-Oriented Programming

1.Problem Statement

The management of residential apartments involves handling multiple types of information such as resident details, apartment allocation, ownership records, and tenancy data. In traditional systems, this information is maintained manually using registers or spreadsheets. These methods are inefficient, difficult to manage, and highly prone to human errors.

One of the major challenges in manual apartment management is the lack of a structured system to clearly differentiate between different types of residents. For example, tenants and owners are often recorded in a similar manner, which creates confusion in managing rent payments and ownership information. This makes the system unreliable and difficult to maintain.

Another significant issue is tracking apartment occupancy. In manual systems, there is no proper mechanism to determine whether an apartment is occupied or vacant at any given time. This leads to mismanagement and incorrect allocation of apartments.

In addition, data redundancy and inconsistency are common problems. Since records are maintained manually, the same information may be entered multiple times, leading to duplication. Updating records is also difficult and may result in errors if not handled carefully.

As the number of residents increases, managing this data becomes increasingly complex. Searching for specific information or updating records requires considerable time and effort. This reduces efficiency and increases the chances of mistakes.

To overcome these limitations, this project proposes an Apartment Management System developed using Object-Oriented Programming (OOP) principles. The system models real-world entities such as residents and apartments into structured classes, enabling efficient and organized data management.

The main objectives of this system are to manage resident data effectively, distinguish clearly between tenants and owners, track apartment occupancy accurately, and demonstrate the practical implementation of OOP concepts.

The system provides a scalable, modular, and efficient solution that improves data organization, reduces redundancy, and ensures better consistency in apartment management.

2.System Design

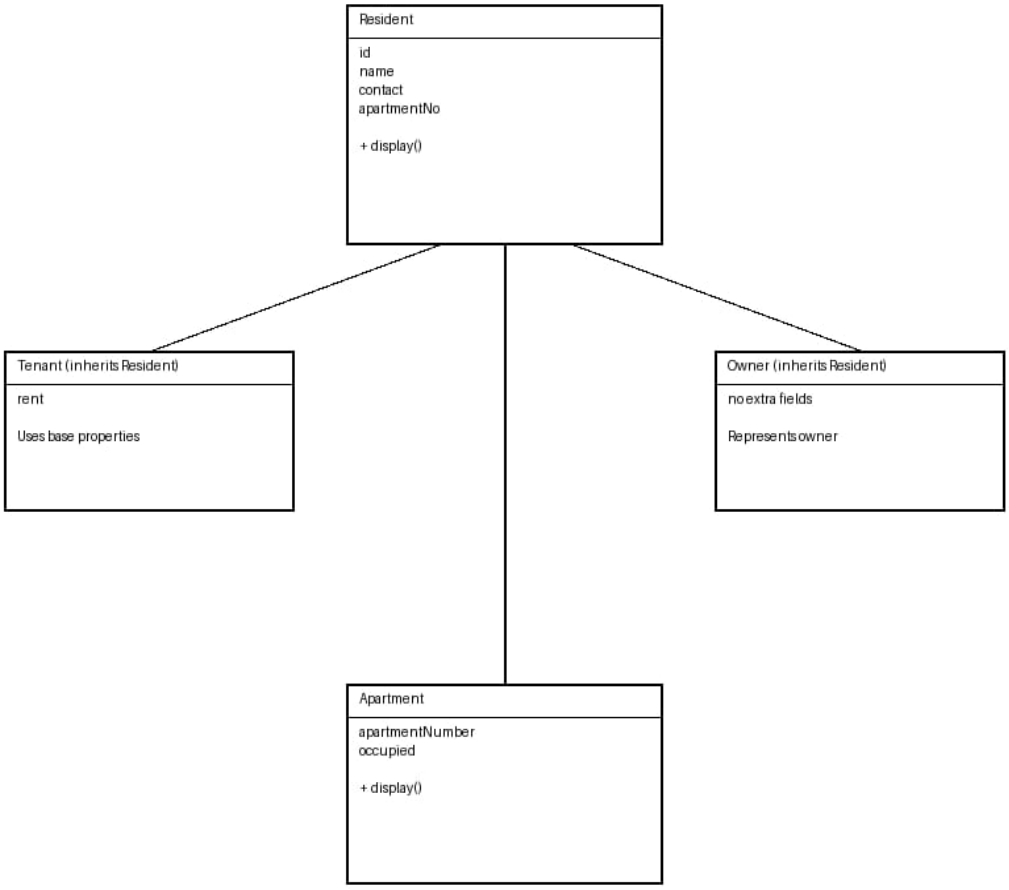
The Apartment Management System is designed using a class-based architecture, where real-world entities are represented as classes. This approach ensures modularity, reusability, and ease

of maintenance. Each class in the system is responsible for handling specific data and operations, which helps in organizing the overall structure efficiently.

The system primarily consists of four main classes: Resident, Tenant, Owner, and Apartment. These classes interact with each other to simulate real-world apartment management.

2.1 UML Class Diagram

It shows Resident as base class, Tenant and Owner inheriting from it, and Apartment associated with Resident.



2.2 Description of Classes

The UML diagram provides a visual representation of the system structure and the relationships between different classes.

At the core of the system is the Resident class, which acts as the base class. It contains common attributes such as id, name, contact, and apartment number. These attributes represent the basic information required for any resident.

The Tenant class and Owner class are derived from the Resident class. This relationship is shown in the UML diagram using inheritance arrows pointing towards the Resident class.

The Tenant class extends the functionality of the Resident class by adding an additional attribute called rent. This attribute stores the monthly rent paid by tenants. By inheriting from the Resident class, the Tenant class can reuse all common properties while adding its own specific feature.

The Owner class also inherits from the Resident class but does not introduce any additional attributes. It represents residents who own their apartments. This demonstrates how inheritance can be used for code reuse without adding unnecessary complexity.

The Apartment class is associated with the Resident class. It contains attributes such as apartment number and occupancy status. The occupancy attribute is typically a boolean value that indicates whether the apartment is currently occupied or vacant.

The association between Apartment and Resident ensures that each apartment can be linked to a resident, allowing the system to track occupancy accurately.

Overall, the UML diagram clearly shows:

Inheritance relationships (Tenant and Owner inheriting from Resident)

Association between Apartment and Resident

Structure and attributes of each class

This design ensures that the system is well-organized, easy to understand, and scalable for future enhancements.

2.3 Description of Major Classes

Resident Class

The Resident class serves as the base class for the system and represents a general resident living in an apartment.

It includes attributes such as id, name, contact, and apartment number. These attributes store essential information about a resident. The class also includes a method called display(), which is used to output the resident’s details.

This class is important because it provides a common structure for all types of residents. Instead of repeating the same attributes in multiple classes, they are defined once and reused through inheritance.

Tenant Class

The Tenant class inherits from the Resident class and represents residents who live in apartments on rent.

In addition to the attributes inherited from the Resident class, the Tenant class includes an extra attribute called rent. This attribute stores the monthly rent paid by the tenant.

This class demonstrates the concept of inheritance by extending the base class and adding its own functionality. It helps in clearly distinguishing tenants from other types of residents.

Owner Class

The Owner class also inherits from the Resident class and represents apartment owners.

Unlike the Tenant class, it does not introduce any new attributes. It simply reuses the properties defined in the Resident class. This shows how inheritance can be used effectively to avoid redundancy.

Apartment Class

The Apartment class is responsible for managing apartment-related information.

It includes attributes such as apartment number and occupancy status. The occupancy status indicates whether the apartment is currently occupied or vacant.

This class is associated with the Resident class and plays a crucial role in linking residents to apartments. It ensures that the system can accurately track apartment availability.

2.4 Package Structure

The project is organized into different packages to improve code readability and maintainability.

The model package contains all the classes such as Resident, Tenant, Owner, and Apartment. These classes represent the core components of the system.

The main package contains the main method, which controls the execution of the program.

This separation of packages ensures better organization and makes the code easier to understand and manage.

2.5 Interaction Flow

The system follows a logical sequence of operations to manage apartment data.

Initially, the user inputs the details of a resident. Based on the input, the system determines whether the resident is a tenant or an owner. After identifying the type, an apartment is assigned to the resident.

Once the apartment is assigned, the occupancy status is updated accordingly. Finally, the system displays the relevant information to the user.

This interaction flow ensures that the system operates smoothly and processes data accurately.

3. OOP Concepts Demonstrated

Object-Oriented Programming (OOP) is the core concept used in this project. The Apartment Management System is designed by applying the four main principles of OOP: Encapsulation, Inheritance, Polymorphism, and Abstraction. These concepts help in creating a modular, scalable, and maintainable system.

3.1 Encapsulation

Encapsulation is the process of binding data and methods together into a single unit and restricting direct access to the data. In this project, encapsulation is implemented by declaring class variables as private and accessing them through public getter and setter methods.

For example, in the Resident class, attributes such as id, name, contact, and apartment number are kept private. These attributes cannot be accessed directly from outside the class. Instead, methods are provided to get and set their values.

The main purpose of encapsulation is to protect data from unauthorized access and modification. It ensures that the internal representation of the data is hidden from the outside world. This improves data security and prevents accidental changes.

Encapsulation also allows validation to be implemented inside setter methods. For instance, a contact number can be validated before storing it in the system. This ensures that only correct and meaningful data is stored.

Overall, encapsulation improves the reliability and maintainability of the system.

3.2 Inheritance

Inheritance is a mechanism that allows one class to acquire the properties and behavior of another class. In this system, inheritance is used to establish a relationship between the Resident class and its derived classes, Tenant and Owner.

The Tenant class extends the Resident class and inherits all its attributes and methods. In addition to the inherited properties, the Tenant class includes an extra attribute called rent. This represents the rent paid by the tenant.

Similarly, the Owner class also extends the Resident class. However, it does not add any new attributes and simply reuses the properties of the base class.

The main advantage of inheritance is that it reduces code duplication. Instead of defining the same attributes multiple times, they are defined once in the base class and reused by derived classes.

Inheritance also improves code organization and makes the system easier to extend. For example, if a new type of resident needs to be added in the future, it can be easily created by extending the Resident class.

3.3 Polymorphism

Polymorphism is the ability of a single entity to take multiple forms. In object-oriented programming, it allows a single reference variable to represent different types of objects.

In this system, polymorphism is demonstrated by using a reference of type Resident to refer to objects of type Tenant and Owner. This means that the same method can be used to perform different actions depending on the type of object.

For example, a Resident reference can point to a Tenant object or an Owner object. When a method such as display() is called, the behavior depends on the actual object being referred to.

Polymorphism increases flexibility and reduces code complexity. It allows the system to handle different types of objects in a uniform way while still supporting specific behaviors.

This makes the system more dynamic and easier to manage.

3.4 Abstraction

Abstraction is the process of hiding internal implementation details and exposing only the necessary features to the user.

In this project, abstraction is used to simplify the interaction between the user and the system. The user does not need to know how the data is stored or processed internally. Instead, the user interacts with simple methods such as display().

Abstraction can be implemented using abstract classes or interfaces. Even when not explicitly defined, the concept is still applied by separating implementation logic from user interaction.

The main advantage of abstraction is that it reduces complexity and improves usability. It allows developers to focus on essential features without worrying about internal details.

4. Implementation Highlights

The Apartment Management System is implemented in Java using object-oriented programming concepts. The system is divided into classes such as Resident, Tenant, Owner, and Apartment, each representing a real-world entity.

Objects are created from these classes, and methods are used to perform operations such as displaying details and managing apartment occupancy.

Code Snippet 1: Resident Class (Encapsulation)

```
class Resident {

    private int id;

    private String name

  public void setId(int id) {

 this.id = id;}

    public int getId() {

        return id;

    }

}
```

```
}
```

Explanation:

The attributes are declared private and accessed using methods, which ensures data security through encapsulation.

Code Snippet 2: Inheritance

```
class Tenant extends Resident {

    private double rent;

}
```

Explanation:

The Tenant class inherits properties from Resident and adds rent, demonstrating inheritance.

Code Snippet 3: Polymorphism

```
resident r;

r=new Tenant();

r.display();

r = new Owner();

r.display();
```

Explanation:

A single reference is used for different object types, showing polymorphism.

Code Snippet 4: Apartment Handling

```
Apartment a = new Apartment();

a.setOccupied(true);
```

Explanation:

This updates the occupancy status of an apartment.

6. Testing & Error Handling

The Apartment Management System is tested using different types of inputs to ensure its correctness and reliability. Testing helps in verifying that the system behaves as expected under various conditions.

Test Cases

The system is tested with both valid and invalid inputs.

Valid input produces correct output and ensures proper functionality

Invalid input is handled gracefully with appropriate error messages

Edge Cases

Special conditions are also considered to ensure robustness of the system:

Empty input fields

Duplicate resident IDs

Assigning an already occupied apartment

These cases help in checking how the system behaves under unusual situations.

Code Snippet: Input Validation

```
If(contact.length() != 10) {
    System.out.println("Invalid contact number");
}
```

Explanation:

This validation ensures that only correct contact numbers are accepted, preventing incorrect data entry into the system.

Error Handling:

The system uses basic validation techniques to prevent incorrect inputs. Instead of crashing, the system displays meaningful error messages to guide the user.

This improves the stability and reliability of the system and ensures smooth execution even when errors occur.

6. Git Workflow

Git was used for version control to track changes throughout the project development.

Repository Setup

↓

Project Initialization

↓

Core Implementation

↓

Code Improvement

↓

Documentation

↓

Presentation (Slides)

↓

Testing & Debugging

↓

Final Submission

Explanation of Workflow

1. Repository Setup

The project repository was created using GitHub Classroom. A proper folder structure was established, including folders for source code, documentation, report, and slides.

2. Project Initialization

Initial commit was made to set up the project environment. Basic files and folders were added to prepare for development.

3. Core Implementation

Classes such as Resident, Tenant, Owner, and Apartment were created. Object-oriented programming concepts like inheritance and encapsulation were applied.

4. Code Improvement

The code was refined by adding comments and improving readability. Naming conventions and structure were enhanced for better understanding.

5. Documentation

The UML diagram was added, and the README file was updated with project details and execution instructions.

6. Presentation (Slides)

Presentation slides were created and uploaded to explain the project during evaluation.

7. Testing & Debugging

The system was tested using different inputs. Errors were identified and corrected to ensure smooth execution.

8. Final Submission

Final commits were made after completing all components of the project, including code, documentation, and slides.

7.Conclusion

The Apartment Management System effectively demonstrates the application of object-oriented programming concepts in a real-world scenario. It provides a structured and efficient way to manage resident and apartment data.

The use of OOP principles makes the system modular, scalable, and easy to maintain. Overall, the project successfully meets its objectives and offers a simple yet reliable solution for apartment management.

8.Future Scope

Graphical user interface can be added using Java Swing.

Database integration can be implemented using MySQL.

Authentication system can be introduced for security.

Online payment system can be integrated.

The system can be expanded into a complete society management system.

. **Appendix**

Apartment Management System – Project Proposal

Project Title

Apartment Management System

Problem Statement

Managing residential apartments manually can lead to difficulties in tracking residents, apartment occupancy, and available units. Building managers often rely on paperwork or informal records, which makes it difficult to maintain accurate information about residents and apartment allocation. This project proposes a simple console-based Apartment Management System developed in Java to assist building administrators in managing apartment data efficiently. The system will allow administrators to add and remove residents, allocate apartments to residents, vacate apartments, and view details of residents and available apartments. By organizing the system using object-oriented programming principles, the application will provide a structured and maintainable solution for managing apartment records. The goal of the project is to demonstrate proper OOP design, modular structure, and clear separation of responsibilities while solving a small real-world management problem.

Target User

- Apartment Building Manager / Administrator

Core Features

- Add new resident
- Remove resident
- Allocate apartment to resident
- Vacate apartment
- View all residents
- View available apartments

OOP Concepts to be Used

- **Abstraction:** Using service interfaces to define apartment management operations.
- **Inheritance:** Base classes can be extended to represent different types of residents or apartments if required.
- **Polymorphism:** Interface implementations will allow flexible behavior of service methods.
- **Exception Handling:** Custom exceptions such as ResidentNotFoundException will be used to handle invalid operations.
- **Collections:** Java Collections (ArrayList) will be used to store and manage resident and apartment data.

Proposed Architecture Description

The system will follow a layered structure separating data models, service logic, and user interaction. Model classes such as Resident and Apartment will represent core entities. A service interface will define apartment management operations, and its implementation class will handle the business logic using Java collections. The Main class will provide a console-based menu interface for user interaction. Custom exception classes will handle error conditions, ensuring clean and maintainable code design.

PCCCS495 Proposal Review – APPROVED_MINOR

S soumadip.biswas@ieminternational.ai

To: Baivabi Chakraborty

Reply Reply all Forward ...

Thu 3/5/2026 10:39 AM

Dear Baivabi Chakraborty ,

Project: Apartment Management System
Decision: APPROVED_MINOR
Score: 90

Strengths:
The proposal clearly defines a practical problem and proposes a suitable console-based solution for a university-level OOP project. The problem justification is strong, highlighting the inefficiencies of manual systems. A comprehensive list of core OOP concepts (Abstraction, Encapsulation, Polymorphism, Inheritance, Exception Handling, Collections) is explicitly mentioned, indicating a strong understanding of the subject matter. The proposed architecture is particularly commendable, advocating for a modular design with clear separation of concerns (Model, Service Interface/Implementation, UI) and the use of custom exception classes. This demonstrates a thoughtful approach to designing maintainable and structured software.

Improvements:
While inheritance is listed as an OOP concept, the proposal does not explicitly detail a scenario within the Apartment Management System where it would be applied. Providing a concrete example (e.g., different types of apartments or residents inheriting from a base class) would strengthen this section. Furthermore, as a 'real-world management problem,' the current architecture, relying solely on Java Collections, would lose all data upon program termination. While not strictly required for a foundational OOP course, acknowledging data persistence (e.g., basic file I/O or serialization) as a future enhancement or briefly explaining why it's out of scope for this phase would enhance the proposal's realism. Lastly, while custom exception handling is mentioned, providing a couple more specific examples of invalid operations beyond 'non-existent resident' that would trigger these exceptions could offer more clarity.

Regards,
Course Instructor

Reply Forward